IN-MEMORY KEY-VALUE STORE (Redis-lite)

Phase 1 — Basic TCP Server (Echo). C++20, POSIX Sockets | What we built, line by line.

1 — WHAT WE ARE BUILDING

The goal

A program that stores data as **key → value** and is reachable **over the network**, so other programs (clients) can connect and run commands.

```
"name" -> "shivam"
"city" -> "surat"
```

Internally it is just an `unordered_map` — exposed over a network.

Phase 1 scope (today)

- Server listens on port **6379**
- One client connects
- Whatever client sends → sent back (**echo**)

No storage. No commands. Only connection + data flow.
Echo first — to prove the connection works before adding GET/SET.

REAL-LIFE — CLOAKROOM
You hand over a bag → get a **TOKEN (#42)**. Show the token → get the bag back. **TOKEN = KEY**, **BAG = VALUE**, counter = the KV store.

2 — SOCKET = A PHONE CALL BETWEEN TWO PROGRAMS

Your `unordered_map` lives *inside* one program. A second program cannot reach inside it. A **socket** is the connection (the "phone line") that lets them talk.

Server = the shop's landline

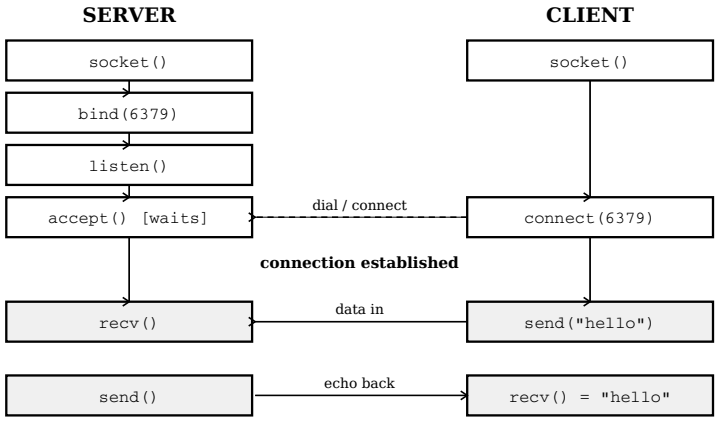
Always ON, waits for calls. Does the 5 steps below.

Client = the caller

Dials the number, then talks. Only 3 steps.

FILE DESCRIPTOR (FD)
In Linux everything is a **NUMBER**. Open a socket → you get a number (the fd). That number is the "handle" you use to listen / send / close. **FD = THE TOKEN THAT LETS YOU ACCESS THE SOCKET.**

3 — THE FLOW (server 5 steps, client 3 steps)



Step	Server	Meaning (real-life)
`socket()`	make endpoint	buy a phone
`bind()`	assign IP + port 6379	give the phone a number
`listen()`	ready for calls	switch phone ON, let it ring
`accept()`	take one connection	pick up the call (waits until one comes)
`recv()/send()`	read / write data	listen / speak

4 — THE CODE, LINE BY LINE

Headers

Line	What it gives
`#include <iostream>`	cout / cerr — printing
`#include <cstring>`	memset — clear a buffer
`#include <sys/socket.h>`	socket, bind, listen, accept, recv, send (POSIX)
`#include <netinet/in.h>`	sockaddr_in — the address struct (IP + port)
`#include <unistd.h>`	close() — release an fd

These are **POSIX** headers (Linux/Unix). This is what makes it real systems code — not Windows Winsock.

1. Create the socket

```
int server_fd = socket(AF_INET, SOCK_STREAM, 0);
if (server_fd == -1) { std::cerr << "socket fail\n"; return 1; }
```

Part	Meaning
`AF_INET`	use IPv4 (which "world" of addresses)
`SOCK_STREAM`	TCP — reliable, ordered (SOCK_DGRAM = UDP)
`0`	protocol = default (picks TCP automatically)
`== -1`	functions return -1 on failure — always check (error handling)
`server_fd`	the socket's fd (its handle/token)

ANALOGY Bought a phone. `SOCK_STREAM` = it's a TCP phone (your reliable kind). `server_fd` = the phone's handle.

2. Fill the address box

```
sockaddr_in address;
memset(&address, 0, sizeof(address));
address.sin_family      = AF_INET;
address.sin_addr.s_addr = INADDR_ANY;
address.sin_port        = htons(6379);
```

Line	Meaning
`sockaddr_in address;`	a struct (box) holding family + IP + port
`memset(...0...)`	wipe the box clean — a fresh struct has garbage; clear before filling
`sin_family = AF_INET`	IPv4 — same as in socket(), stay consistent
`s_addr = INADDR_ANY`	listen on ANY of this machine's IPs
`sin_port = htons(6379)`	port 6379, converted to network byte order

HTONS() — WHY?
CPU and network store numbers in different byte order. `htons` = "Host TO Network Short" = translate the port so the other machine reads it correctly. **ALWAYS WRAP THE PORT IN HTONS.**
MEMSET ANALOGY: clear/blank a new form fully before writing your own values on it.

3. Bind — attach the address to the socket

```
if (bind(server_fd, (sockaddr*)&address, sizeof(address)) == -1) {
    std::cerr << "bind fail — port busy?\n"; return 1;
}
```

Arg	Meaning
`server_fd`	which socket gets the address
`(sockaddr*)&address`	the address box, cast to the generic type bind expects
`sizeof(address)`	how big that box is

Fails (`-1`) if the port is already in use by another program. The cast `(sockaddr*)` is a required formality — bind takes a generic `sockaddr`.

ANALOGY The phone now has its number (6379). Built, numbered — but still OFF.

4. Listen + Accept

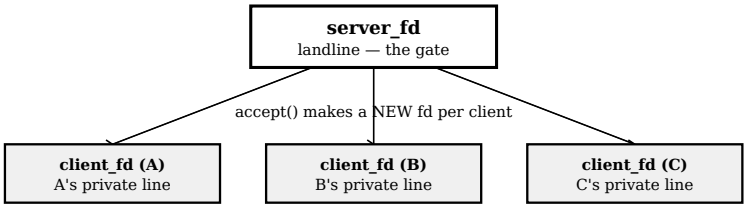
```
listen(server_fd, 5);                 // turn ON, allow 5 waiting
```

```
int client_fd = accept(server_fd, nullptr, nullptr);  // pick up
```

Part	Meaning
`listen(fd, 5)`	socket goes live; "5" = backlog (5 callers can wait in line)
`accept(server_fd...)`	blocks (waits) until a client connects; returns a NEW fd
`client_fd`	that new fd — the private line to THIS client
`nullptr, nullptr`	"don't need the client's address right now"

ANALOGY `listen` = phone ON, ringing allowed; backlog = waiting line. `accept` = pick up the call (you wait here until someone calls).

KEY IDEA — server_fd vs client_fd



recv() / send() happen on the private lines — NOT on server_fd

one server_fd · a separate client_fd per connection

PIZZA SHOP The landline (server_fd) only connects calls. Each customer gets a private line (client_fd) where the real order happens. The landline stays free for the next caller.

5. Echo loop — recv then send back

```
char buffer[1024];
while (true) {
    memset(buffer, 0, sizeof(buffer));
    int bytes = recv(client_fd, buffer, sizeof(buffer), 0);
    if (bytes <= 0) { break; }            // client left
    send(client_fd, buffer, bytes, 0);    // echo back
}
```

Line	Meaning
`char buffer[1024]`	a 1024-byte box to hold incoming data (a notepad)
`while (true)`	client may send many messages — keep listening
`memset(buffer,0,..)`	clear old data before each read
`recv(client_fd,...)`	read from the PRIVATE line into buffer; returns bytes read; blocks until data
`bytes <= 0`	0 = client closed, <0 = error → break the loop
`send(client_fd, buffer, bytes, 0)`	send back exactly "bytes" (only what arrived, not full 1024)

WHY `BYTES`, NOT `SIZEOF(BUFFER)`? Client sent 5 bytes ("hello"). Send back 5 — not 5 real + 1019 garbage. Echo = exactly what came in.

6. Cleanup

```
close(client_fd);   // close the client line
close(server_fd);   // close the landline
return 0;
```

`close()` frees the fd. Done with work → close both, so resources don't leak.

5 — COMPILE & RUN (WSL / Linux)

Terminal 1 — server

```
g++ -std=c++20 main.cpp -o server
./server
```

Output:

```
Server ready on port 6379...
```

Cursor pauses here = correct. It is stuck at `accept()`, waiting for a client.

Terminal 2 — client

```
nc localhost 6379
hello shivam        <- you type
hello shivam        <- server echoes
```

Server side prints `Client connected!`
(If nc missing: `sudo apt install netcat-openbsd -y`)

Why "hello" shows twice: first line = your typing; second = the server's echo. **Stop:** Ctrl+C in each terminal.

6 — DONE / ROADMAP

Phase 1 — verified working

- socket → bind → listen → accept
- recv → send (echo)
- server_fd vs client_fd understood
- compiled with g++, tested with nc

Next phases

- **2 — Commands:** parse GET / SET / DEL
- 3 — Storage + thread safety (hashmap + sharded locks)
- 4 — TTL (auto-expiry)
- 5 — LRU eviction
- 6 — Benchmark (50K+ ops/sec)

IN-MEMORY KEY-VALUE STORE (Redis-lite)

Phase 2 + Phase 3 — Command Parsing & Storage. C++20, POSIX Sockets | Files, relations, line-by-line, data structures.

1 — PROJECT STRUCTURE NOW (after modular refactor)

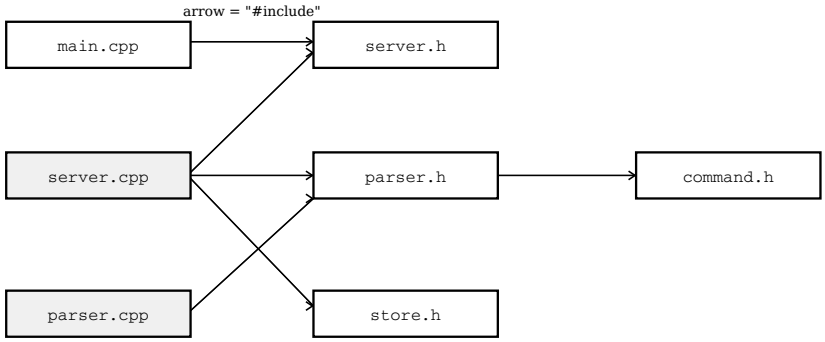
```
kvstore/
├── CMakeLists.txt      build config
├── include/            headers (.h) — "what"
│   ├── command.h       Command struct + enum
│   ├── parser.h        parse() declaration
│   ├── store.h         KVStore class declaration
│   └── server.h        Server class declaration
└── src/                sources (.cpp) — "how"
    ├── main.cpp        entry point (6 lines)
    ├── server.cpp      socket + loop + dispatch
    ├── parser.cpp      tokenize + validate
    └── store.cpp       set / get / del
```

What each file does

- **main.cpp** — makes a Server, runs it
- **server.h/.cpp** — TCP socket, accept, recv/send loop, command dispatch
- **command.h** — the parsed-command data shape (struct + enum)
- **parser.h/.cpp** — turn a raw string into a Command
- **store.h/.cpp** — the actual key→value storage (unordered_map)

.H VS .CPP .h = MENU (what exists). .cpp = KITCHEN (how it's made).

2 — FILE RELATIONS (who includes whom)



File	includes	why
main.cpp	server.h	needs the Server class
server.cpp	server.h, parser.h, store.h	runs socket, parses input, stores data
parser.cpp / parser.h	command.h	returns a Command struct
store.cpp	store.h	implements the KVStore class

3 — PHASE 2 : COMMAND PARSING

Goal: turn a raw string the client sent into a structured command the server can act on. (Understanding only — no storage yet.)

3.1 — command.h : the data shape

```
enum class CommandType { SET, GET, DEL, UNKNOWN };

struct Command {
    CommandType type  = CommandType::UNKNOWN;
    std::string key;
    std::string value;
    bool        valid = false;
    std::string error;
};
```

Part	Meaning
`enum class CommandType`	fixed list of allowed commands — type-safe (cannot be a random value)
`type = UNKNOWN`	which command; default UNKNOWN (safe)
`key / value`	the parsed key and value (value only for SET)
`valid = false`	did parsing succeed? default false (safe)
`error`	message when parsing fails

ORDER SLIP The customer speaks; the receptionist writes a slip: item, variant, size. The slip = the Command struct.

"SET name shivam" →

```
type  : SET
key   : "name"
value : "shivam"
valid : true
error : ""
```

"SET name" (bad) →

```
type  : UNKNOWN
key   : ""
value : ""
valid : false
error : "SET needs key and value"
```

3.2 — parser.h : the declaration

```
Command parse(const std::string& input);
```

Part	Meaning
`Command`	function returns a Command struct
`const std::string& input`	takes the raw text by reference (no copy) and promises not to change it
`; (no body)`	declaration only — real code lives in parser.cpp

CONST T& Pass the book itself (fast, no photocopy) with a promise "I won't write on it." Standard C++ for read-only inputs.

3.3 — parser.cpp : the logic (line by line)

```
Command parse(const std::string& input) {
    Command cmd;
    std::stringstream ss(input);
    std::string token;
    std::vector<std::string> tokens;
    while (ss >> token) tokens.push_back(token);

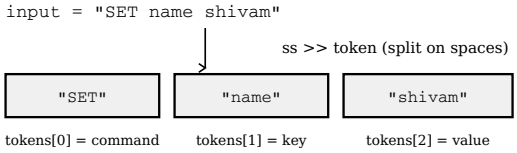
    if (tokens.empty()) { cmd.error = "empty command"; return cmd; }
    const std::string& cmdName = tokens[0];

    if (cmdName == "SET") {
        if (tokens.size() != 3) { cmd.error="SET needs key and value"; return cmd; }
        cmd.type=CommandType::SET; cmd.key=tokens[1]; cmd.value=tokens[2];
        cmd.valid=true; return cmd;
    }
    if (cmdName == "GET") { /* needs size 2 */ ... }
    if (cmdName == "DEL") { /* needs size 2 */ ... }
    cmd.error = "unknown command: " + cmdName;
    return cmd;
}
```

Line	Meaning
`stringstream ss(input)`	treat the string like an input stream; splits on whitespace
`while (ss >> token)`	pull one word at a time until empty
`tokens.push_back(token)`	store each word in a vector (dynamic array)
`tokens.empty()`	nothing typed → error, return

`cmdName = tokens[0]`	first word = the command name
`size != 3 / != 2`	validate token count per command
`cmd.valid = true`	mark success only after validation passes

tokenize visualization



3.4 — server.cpp : the parse loop (recv → parse → respond)

```
int bytes = recv(client_fd, buffer, sizeof(buffer), 0);
if (bytes <= 0) break;
std::string input(buffer, bytes);                 // bytes only, not 1024
while (!input.empty() && (input.back()=='\n'||input.back()=='\r'))
    input.pop_back();                              // trim newline from nc
Command cmd = parse(input);
// build response from cmd, then:
send(client_fd, response.c_str(), response.size(), 0);
```

Line	Meaning
`string input(buffer, bytes)`	build a string from the first *bytes* chars only (the rest of the 1024 are zero)
`trim \n / \r`	nc appends Enter; remove it or the last token becomes "shivam\n"
`parse(input)`	call the parser → Command struct back
`response.c_str(), .size()`	send() needs a char* + length; convert the string

4 — PHASE 3 : STORAGE (the real Redis behaviour)

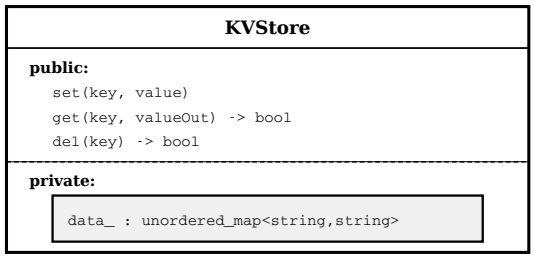
Goal: actually store/fetch/delete data in an `unordered_map`, wrapped in a KVStore class.

4.1 — store.h : the KVStore class

```
class KVStore {
public:
    void set(const std::string& key, const std::string& value);
    bool get(const std::string& key, std::string& valueOut);
    bool del(const std::string& key);
private:
    std::unordered_map<std::string,std::string> data_;
};
```

Member	Meaning
`set(key, value)`	store or update; returns nothing
`get(key, valueOut)`	returns bool (found?) and fills **valueOut** with the value
`del(key)`	returns bool (was something deleted?)
`private data_`	the actual map — hidden; only reachable through set/get/del (encapsulation)

GET() OUT-PARAMETER A function returns ONE thing, but get must answer two: "found?" and "the value." So: return the bool, and write the value into `valueOut` (a reference = the caller's own variable). You hand the receptionist YOUR box (&); they fill it and say "yes."



4.2 — store.cpp : set / get / del (line by line)

```
void KVStore::set(const std::string& key, const std::string& value) {
    data_[key] = value;                      // insert OR overwrite
}
bool KVStore::get(const std::string& key, std::string& valueOut) {
    auto it = data_.find(key);               // safe lookup
    if (it == data_.end()) return false;     // not found
    valueOut = it->second;                    // write value to caller's var
    return true;
}
bool KVStore::del(const std::string& key) {
    return data_.erase(key) > 0;             // erase returns count removed
}
```

Line	Meaning
`data_[key] = value`	if key absent → create; if present → overwrite
`data_.find(key)`	look up WITHOUT creating a blank entry (unlike `data_[key]`)
`it == data_.end()`	end() is the "not found" marker
`it->second`	iterator points to a pair: first = key, second = value
`valueOut = it->second`	fill the caller's variable (the out-parameter)
`erase(key) > 0`	erase returns how many removed: 1 = deleted, 0 = wasn't there

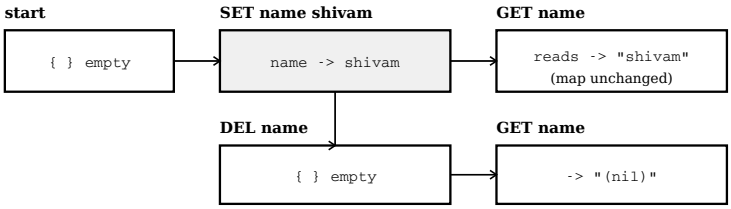
FIND VS [ ] `data_[key]` on a missing key silently creates a blank entry — a bug for GET. `find()` only looks; it never creates.

4.3 — server.cpp : dispatch (the switch that uses KVStore)

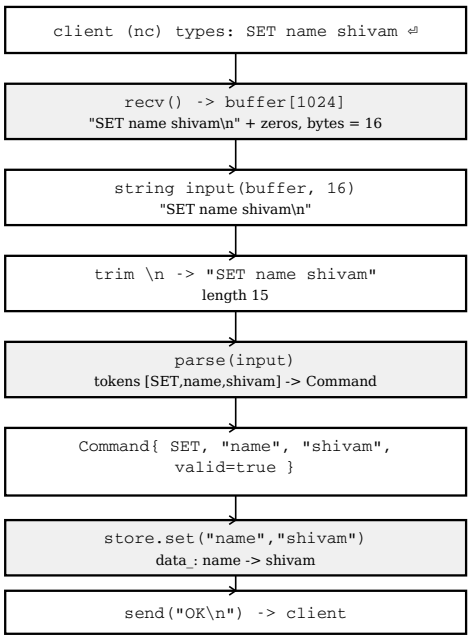
```
KVStore store;                       // ONE store, created before the loop
switch (cmd.type) {
  case SET: store.set(cmd.key, cmd.value); response = "OK\n"; break;
  case GET: { std::string v;
              response = store.get(cmd.key, v) ? v + "\n" : "(nil)\n"; break; }
  case DEL: response = store.del(cmd.key) ? "OK\n" : "(nil)\n"; break;
}
```

`KVStore store;` is declared OUTSIDE the loop so data survives between commands. Here the **valueOut** finally gets used: `store.get(cmd.key, v)` fills `v`, then `response = v`.

data_ state through SET / GET / DEL

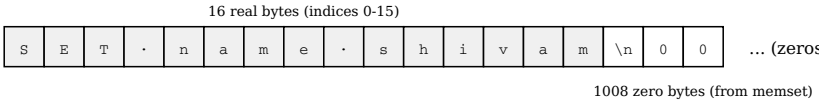

start
{ } empty
SET name shivam
name -> shivam
GET name
reads -> "shivam"
(map unchanged)
DEL name
{ } empty
GET name
-> "(nil)"

5 — END-TO-END EXECUTION FLOW : "SET name shivam"



6 — DATA STRUCTURE VISUALIZATIONS

6.1 — buffer[1024] right after recv (16 real bytes)



`sizeof(buffer)` = 1024 (size in bytes). `bytes` = 16 (what recv actually received). We build the string from **bytes**, not 1024.

6.2 — Command struct (filled)

```
Command
+-----------------------------+
| type   : CommandType::SET   |
| key    : "name"             |
| value  : "shivam"           |
| valid  : true               |
| error  : ""                 |
+-----------------------------+
```

unordered_map data_

```
+---------+-----------+
|  KEY    |  VALUE    |
+---------+-----------+
| "name"  | "shivam"  |
| "city"  | "surat"   |
+---------+-----------+
 find(key) -> iterator
   it->first  = key
   it->second = value
 end() = "not found"
```

7 — STATUS / ROADMAP

Done

- Phase 1 — TCP server (echo)
- Phase 2 — Command parsing (SET/GET/DEL + errors)
- Phase 3 — Storage (real SET/GET/DEL via unordered_map)
- Modular refactor: include/ + src/, CMake

Next (the "systems" depth)

- **Phase 4 — TTL** (background expiry thread)
- **Phase 5 — LRU** eviction (O(1) map + list)
- **Phase 6 — Concurrency**: many clients, sharded locking, P50/P99 + 50K ops/sec benchmark

Phases 1-3 are the foundation. Phases 4-6 are what stand out.

build & run

```
g++ -std=c++20 -Iinclude src/main.cpp src/server.cpp src/parser.cpp src/store.cpp -o server
./server                    # terminal 1
nc localhost 6379           # terminal 2
  SET name shivam  -> OK
  GET name         -> shivam
  GET city         -> (nil)
  DEL name         -> OK
  GET name         -> (nil)
```

IN-MEMORY KEY-VALUE STORE (Redis-lite)

Phase 4 — TTL : Lazy + Active Expiry. C++20 | chrono, background thread, mutex, atomic.

1 — WHAT IS TTL

The idea

`EXPIRE name 3` → the key "name" auto-deletes after 3 seconds. Before TTL, a SET key lived forever.

```
SET name shivam   -> permanent
EXPIRE name 3     -> dies at now+3s
GET name (later)  -> (nil)
```

MILK CARTON A "use by" date. Date passes → spoiled → throw it out. The key's expiry timestamp is that date.

Store value + when-it-dies

```
"name" -> { value: "shivam",
            expireAt: 12:00:05,
            hasExpiry: true }
```

Two ways to remove a dead key

Strategy	How	Trade-off
`LAZY (passive)`	check only when GET touches the key; if dead, delete then	simple, no thread; BUT a key nobody GETs stays in memory
`ACTIVE`	a background thread scans the whole map every 1s and deletes dead keys	memory stays clean; BUT needs a thread + locking

Real Redis uses BOTH. We do too — lazy in `get()`, active in `cleanupLoop()`.

SHOPKEEPER LAZY = check the milk only when a customer asks for it. ACTIVE = walk the fridge every hour and toss spoiled milk yourself.

2 — store.h : Entry struct + new members

```
struct Entry {
    std::string value;
    std::chrono::steady_clock::time_point expireAt;
    bool hasExpiry = false;
};

class KVStore {
public:
    KVStore();   ~KVStore();                 // start / stop the thread
    void set(...); bool get(...); bool del(...);
    bool expire(const std::string& key, int seconds);   // NEW
private:
    void cleanupLoop();                       // the thread's job
    std::unordered_map<std::string, Entry> data_;
    std::mutex mutex_;                        // lock for data_
    std::thread cleaner_;                     // background thread
    std::atomic<bool> running_;               // keep looping?
};
```

Member	Meaning
`Entry.expireAt`	a time_point — when this key dies
`Entry.hasExpiry`	does this key even have a TTL? false = permanent (default)
`steady_clock`	a clock that only moves forward; user can't change it (unlike system clock) — reliable for expiry
`mutex_`	the lock that protects data_ from two threads at once
`cleaner_`	the background thread object
`running_ (atomic)`	a single bool, read by the loop, set false by destructor — atomic fits a single value

ATOMIC VS MUTEX — DESIGN CHOICE `running_` is one bool → **ATOMIC**. `data_` is a whole map → **MUTEX** (you cannot make a map atomic). Right tool for each.

3 — store.cpp : the logic

3.1 — expire() : set a deadline

```
bool KVStore::expire(const std::string& key, int seconds) {
    std::lock_guard<std::mutex> lock(mutex_);
    auto it = data_.find(key);
    if (it == data_.end()) return false;          // no key -> nothing to expire
    it->second.expireAt  = steady_clock::now() + std::chrono::seconds(seconds);
    it->second.hasExpiry = true;
    return true;
}
```

Line	Meaning
`now() + seconds(n)`	current time + an n-second duration = a future time_point (the deadline)
`std::chrono::seconds(n)`	turns the plain int n into a duration you can add to a time_point
`hasExpiry = true`	now GET will start checking this key

3.2 — get() : LAZY expiry

```
bool KVStore::get(const std::string& key, std::string& valueOut) {
    std::lock_guard<std::mutex> lock(mutex_);
    auto it = data_.find(key);
    if (it == data_.end()) return false;
    Entry& e = it->second;
    if (e.hasExpiry && steady_clock::now() > e.expireAt) {
        data_.erase(it);                          // dead -> delete now
        return false;                             // act like it never existed
    }
    valueOut = e.value;
    return true;
}
```

Part	Meaning
`Entry& e = it->second`	reference (not a copy) so erase actually affects the real map
`e.hasExpiry && now > e.expireAt`	has a TTL AND deadline passed → dead
`&& short-circuit`	permanent key (hasExpiry=false) → the time check is skipped entirely (efficient)
`erase(it); return false`	delete the dead key right here, tell caller "(nil)"

4 — THE BACKGROUND THREAD (active expiry)

4.1 — constructor / destructor : start & stop

```
KVStore::KVStore() : running_(true) {
    cleaner_ = std::thread(&KVStore::cleanupLoop, this);   // START thread
}
KVStore::~KVStore() {
    running_ = false;                 // tell the loop to stop
    if (cleaner_.joinable()) cleaner_.join();   // wait for it to finish
}
```

Part	Meaning
constructor runs when?	automatically when `KVStore store;` is created (in server.cpp)
`std::thread(&fn, this)`	run cleanupLoop on a SEPARATE thread (NOT a normal call — main does not block)
destructor runs when?	automatically when the store goes out of scope (run() ends)
`running_ = false`	flips the flag → the while loop exits
`join()`	main waits until the thread fully stops, else it would touch a dead object → crash

call vs thread `cleanupLoop();` would block main forever (it has while(true)). `std::thread(...)` puts it on its own track so main keeps serving clients while the thread cleans — both run at the same time.

4.2 — cleanupLoop : sleep, scan, erase

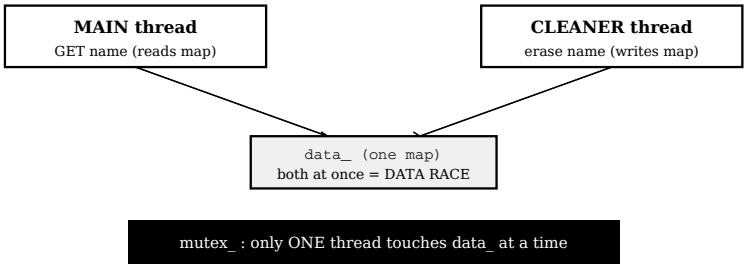
```
void KVStore::cleanupLoop() {
    while (running_) {
        std::this_thread::sleep_for(std::chrono::seconds(1));   // wait 1s
        std::lock_guard<std::mutex> lock(mutex_);                // lock AFTER sleep
        auto now = steady_clock::now();
        for (auto it = data_.begin(); it != data_.end(); ) {
            if (it->second.hasExpiry && now > it->second.expireAt)
                it = data_.erase(it);     // erase returns the NEXT iterator
            else
                ++it;
```

```
        }
    }
}
```

Line	Meaning
`while (running_)`	loop until destructor sets running_=false
`sleep_for(seconds(1))`	scan once per second (don't burn CPU continuously)
`lock AFTER sleep`	don't hold the lock while sleeping, or main is blocked for a whole second
`it = data_.erase(it)`	erasing invalidates the iterator; erase returns the next valid one — so DON'T ++it here
`else ++it`	only advance manually when you didn't erase

Classic gotcha Deleting from a map inside a loop with `++it` in the for-header crashes (the iterator is dead after erase). Pattern: `it = erase(it)` on delete, `++it` otherwise. The for-header has no `++it`.

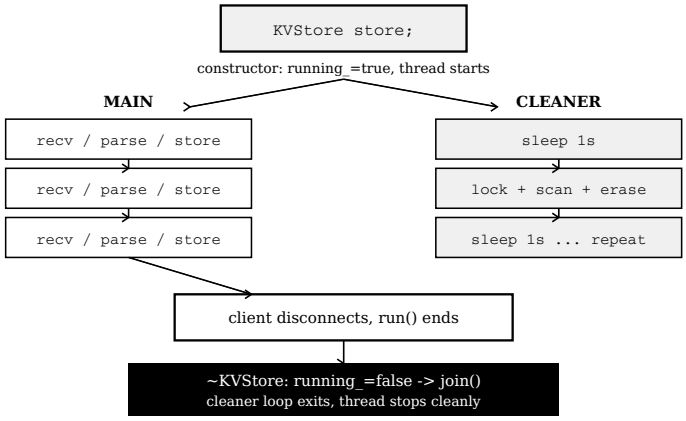
5 — WHY THE MUTEX : data race



Concept	Meaning
`data race`	two threads touch the same map at the same time (one writing) → corruption / crash
`std::mutex`	a lock; whoever holds it owns data_; others wait
`lock_guard<mutex> lock(mutex_)`	locks on creation, auto-unlocks at end of scope (RAII) — no manual unlock to forget
`where`	every function that touches data_ : set, get, del, expire, cleanupLoop

ONE PEN ON THE REGISTER Only the holder may read/write. The cleaner waits until you put the pen down — they never clash.

6 — THREAD LIFECYCLE (main + cleaner run together)



7 — DRY RUN (time starts 12:00:00)

Time	Command	data_	Reply
00	`SET name shivam`	name {shivam, -, exp:false}	OK
02	`EXPIRE name 5`	name {shivam, 12:00:07, exp:true}	OK
04	`GET name`	alive (4 < 7)	shivam
09	`GET name`	{ } — LAZY delete (9 > 7)	(nil)

Active path (no GET at all)

```
SET test hello ; EXPIRE test 3 ; (wait, do NOT GET)
... 3s later, cleaner thread:
   [cleaner] expired key removed: test       <- printed on SERVER terminal
GET test  ->  (nil)   (already gone)
```

8 — STATUS / BUILD

Done

- Phase 1 — TCP server
- Phase 2 — Parsing
- Phase 3 — Storage
- **Phase 4 — TTL (lazy + active), mutex, thread, atomic**

Next

- **Phase 5 — LRU** eviction (memory cap)
- **Phase 6 — Concurrency**: many clients, sharded locking, P50/P99 + 50K ops/sec

Phase 4 is the first real "systems" milestone — threads & locks.

```
g++ -std=c++20 -Iinclude -pthread src/main.cpp src/server.cpp src/parser.cpp src/store.cpp -o server
# -pthread is required once threads are used (else: undefined reference to pthread_create)
```

LRU EVICTION — Phase 5

The map, the list, and the iterator (listIt) — explained for a total beginner.

1 — WHY LRU? (and why not TTL?)

TTL deletes keys by **TIME**. But most keys are permanent (no EXPIRE). If memory fills with permanent keys, TTL does nothing — the server would crash. **LRU** fixes this: when full, throw out the key used **longest ago**.

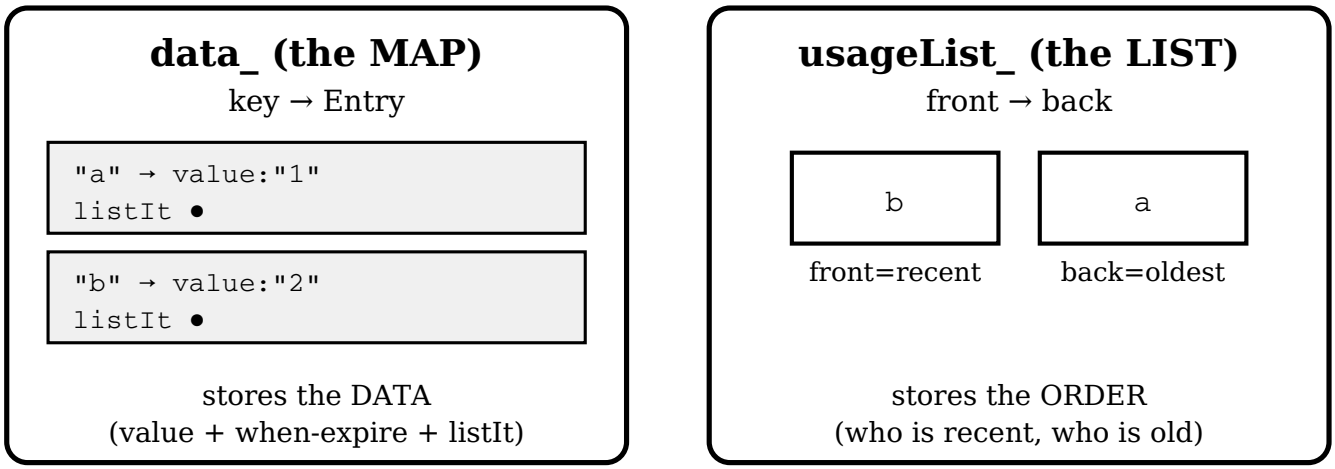
> **SMALL ALMIRAH (CUPBOARD)**
> You own a cupboard that holds only 2 boxes. A new box arrives but the cupboard is full. You throw out the box you touched
> **LEAST RECENTLY**
> — and put the new one in. That "throw out the oldest-used" rule
> **IS**
> LRU.

TTL	LRU
removes by **time** (EXPIRE set)	removes by **space** (memory full)
only expiry keys	any key, even permanent
trigger: timer / GET	trigger: SET when full

They do **different jobs**. Real Redis uses both. So do we.

2 — THE TWO CONTAINERS (this is the whole idea)

LRU needs **TWO** things working together:

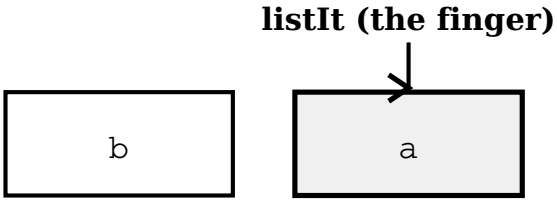


Container	Stores	Job
`data_ (map)`	key → value, expiry, listIt	the actual DATA + fast lookup
`usageList_ (list)`	just the keys, in order	track recent (front) vs old (back)

3 — THE ITERATOR (listIt) — READ THIS SLOWLY

An iterator is just a FINGER pointing at one box in the list.

> **A FINGER ON A SHELF**
>
> The list is a row of boxes on a shelf: [b][a]. An
>
> **ITERATOR**
>
> is your finger resting on ONE box. "This one." That's all it is. `listIt` is a finger pointing at where a key sits in the list.

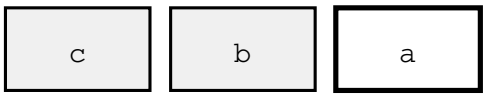


"a" is right here — no need to search the whole list

WHY do we keep this finger? (the key insight)

Without the finger, to find "a" in the list you must check every box one by one — **slow (O(n))**. With the finger stored in the map, you jump straight to "a" — **instant (O(1))**.

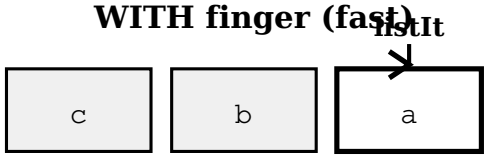
WITHOUT finger (slow)



check c? no. check b? no. check a? yes.

3 steps — O(n)

WITH finger (fast)



jump straight to "a"

1 step — O(1)

listIt = the key's saved finger into the list.
It is what makes LRU O(1) instead of O(n).

In code, the finger lives inside each Entry in the map:

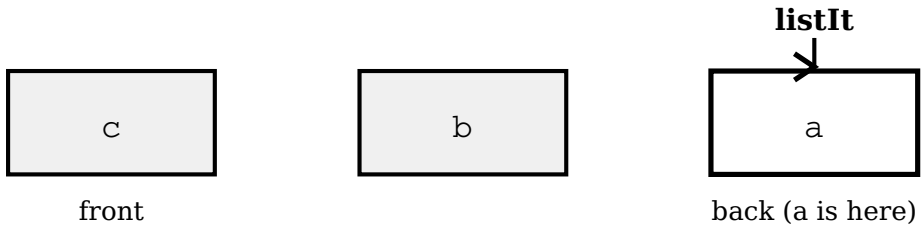
```
struct Entry {
    std::string value;        // the data
    ...
    std::list<std::string>::iterator listIt;   // <-- the FINGER into usageList_
};
```

4 — touch("a") — MOVE A KEY TO FRONT, STEP BY STEP

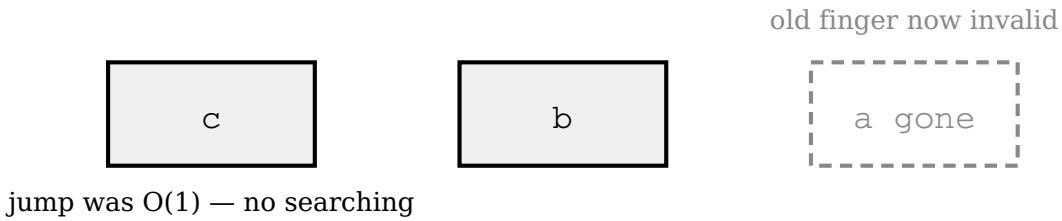
When a key is used (GET or SET), we move it to the **front** (most recent). This is `touch()`. Watch the finger the whole time.

```
void touch(key) {
    auto it = data_.find(key);              // find the key in the map
    usageList_.erase(it->second.listIt);    // STEP 1: finger -> remove that node
    usageList_.push_front(key);             // STEP 2: add key at the front
    it->second.listIt = usageList_.begin(); // STEP 3: save the NEW finger
}
```

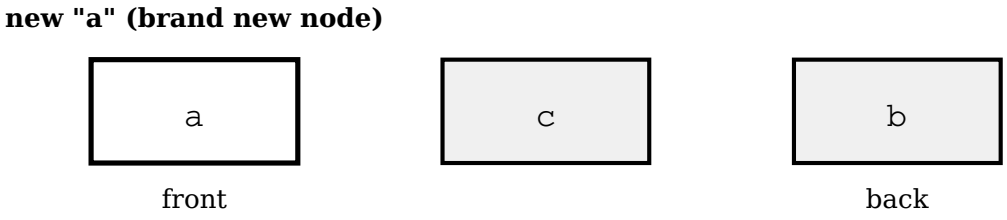
START — list is [c, b, a], we call touch("a")



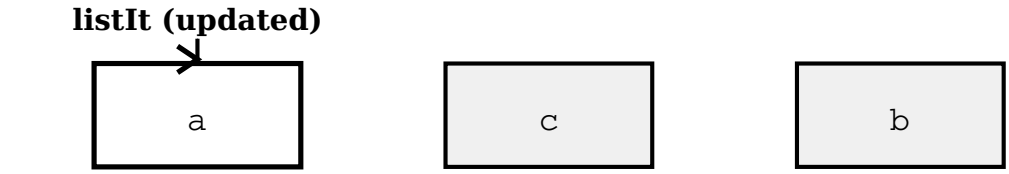
STEP 1 — erase(listIt): finger jumps to "a", removes it → list [c, b]



STEP 2 — push_front("a"): new "a" node at the front → list [a, c, b]



STEP 3 — listIt = begin(): save new finger at front → ready, still O(1)



finger now points at front "a"

WHY UPDATE THE FINGER (STEP 3)?

The old "a" box was thrown away; a brand-new "a" box now sits at the front. The old finger pointed at empty air. We must move the finger to the new box — otherwise next time it points at nothing (crash). `begin()` = the front box = the new "a".

5 — FULL DRY RUN (capacity = 2) — MAP + LIST together

Capacity is 2. Watch BOTH the map and the list after every command. The last SET causes an eviction.

SET a 1 → store "a"

MAP data_

```
"a" → {"1"}
```

LIST usageList_

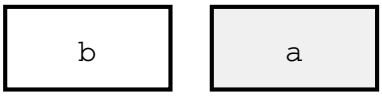


SET b 2 → store "b" (now FULL: 2 keys)

MAP data_

```
"a" → {"1"}
"b" → {"2"}
```

LIST usageList_



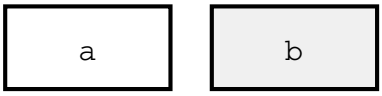
front=recent back=oldest

GET a → return "1", then touch("a") → "a" goes to front

MAP data_ (unchanged)

```
"a" → {"1"}
"b" → {"2"}
```

LIST (a moved to front!)



recent (a) oldest (b) ← danger

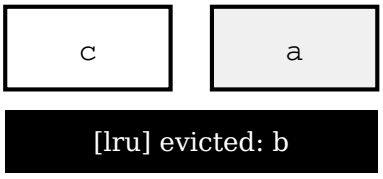
Important: now "b" is the oldest (back). Nobody touched "b" since it was added.

SET c 3 → add "c", now 3 keys > capacity 2 → EVICT the back = "b"

MAP data_

```
"c" → {"3"}
"a" → {"1"}
"b" → REMOVED
```

LIST



Command	List after	Map after	Reply
`SET a 1`	[a]	{a}	OK
`SET b 2`	[b, a]	{a, b}	OK

`GET a`	[a, b]	{a, b}	1
`SET c 3`	[c, a]	{a, c}	OK + evict b
`GET b`	[c, a]	{a, c}	(nil)

Why b and not a? Because `GET a` made "a" recent. "b" was never touched again → it became the oldest → it got evicted.

6 — THE GOLDEN RULE + STATUS

Map and List must ALWAYS stay in sync

Every key lives in **both** the map and the list. So:

- **Add** a key → push to list front **AND** insert in map
- **Use** a key (GET/SET) → touch() moves it to list front **AND** updates its listIt
- **Remove** a key (evict / del / expire) → erase from list **AND** erase from map

If you remove from only one, the other holds a dead entry → bug/crash. Always BOTH.

Done	Next
Phase 1 TCP · Phase 2 Parsing · Phase 3 Storage · Phase 4 TTL · **Phase 5 LRU**	**Phase 6** — Concurrency (many clients, sharded locking) + Benchmark (50K+ ops/sec)

```
g++ -std=c++20 -Iinclude -pthread src/main.cpp src/server.cpp src/parser.cpp src/store.cpp -o server
# test with: KVStore store(2);  then SET a/b, GET a, SET c  ->  b is evicted
```

LRU DEEP DRY RUN — map ↔ listIt ↔ list

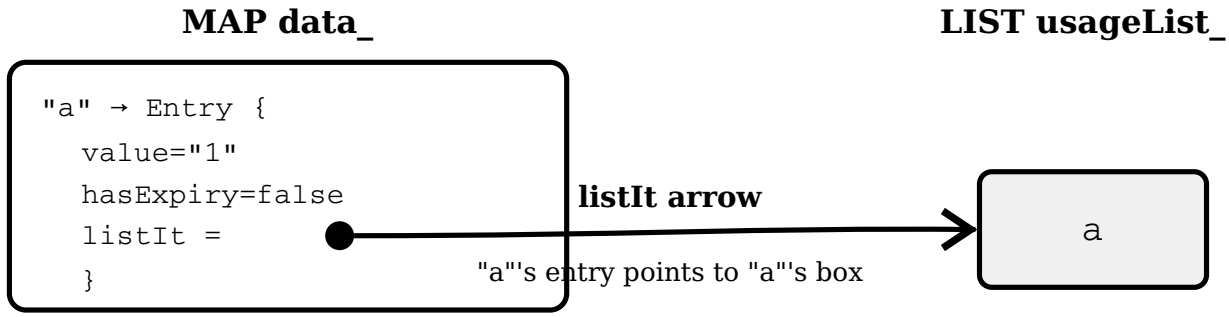
Every step. Every field. The arrow from the map's listIt to the list node is the whole secret. (capacity = 2)

0 — THE RELATION (understand this first)

Each key in the **MAP** stores an **Entry** with 4 fields:

```
struct Entry {
    std::string  value;       // the data, e.g. "1"
    time_point   expireAt;    // when it dies (only if TTL set)
    bool         hasExpiry;   // does it have a TTL? false = permanent
    list::iterator listIt;    // <-- ARROW pointing to this key's node in the LIST
};
```

The **LIST** holds only the keys, in recency order. The `listIt` field is an **arrow from the map entry to that key's box in the list**. That arrow is the link between the two containers.

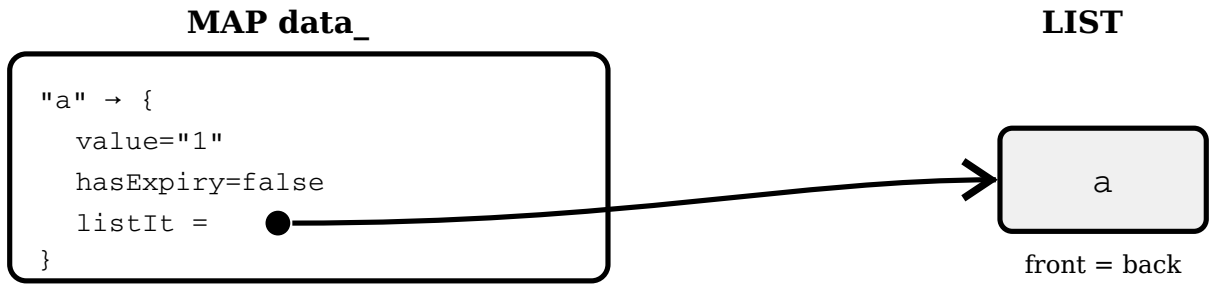


> **HOTEL REGISTER & ROOMS**
> The MAP is the front-desk register: each guest's name → their details +
> **THEIR ROOM NUMBER**
> . The LIST is the actual row of rooms. `listIt` = the room number written next to the guest — so you walk straight to their room, never searching every door.

DRY RUN — capacity = 2

STEP 1 — SET a 1

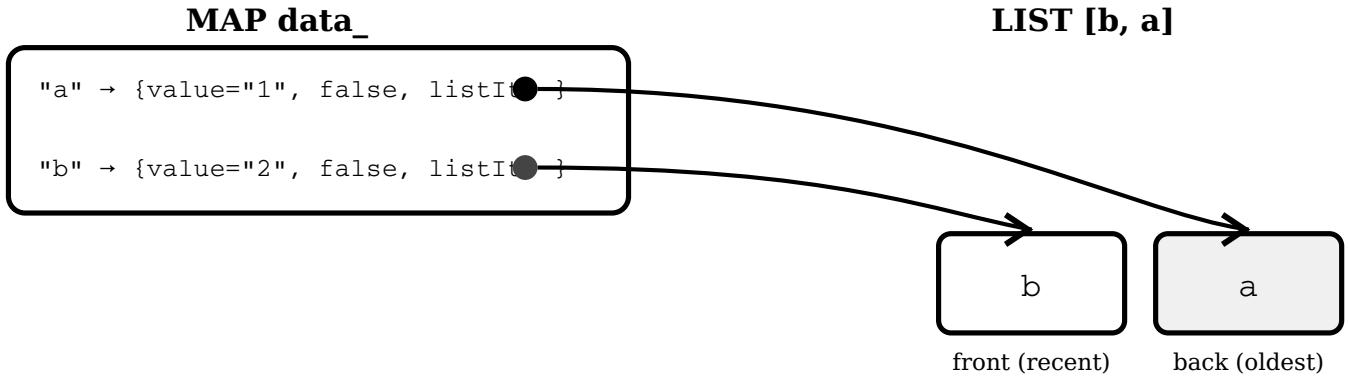
```
// new key path inside set():
usageList_.push_front("a");        // list becomes [a]
Entry e;
e.value = "1";
e.hasExpiry = false;               // permanent
e.listIt = usageList_.begin();     // arrow -> the "a" box (front)
data_["a"] = e;
evictIfNeeded();                   // size 1 <= 2 -> nothing
```



Result: map has "a" (value 1, permanent, listIt→a). List = [a]. Reply: `OK`

STEP 2 — SET b 2 (list now full: 2 keys)

```
usageList_.push_front("b");        // list becomes [b, a]
Entry e; e.value = "2"; e.hasExpiry = false;
e.listIt = usageList_.begin();     // arrow -> the "b" box (front)
data_["b"] = e;
evictIfNeeded();                   // size 2 <= 2 -> nothing
```



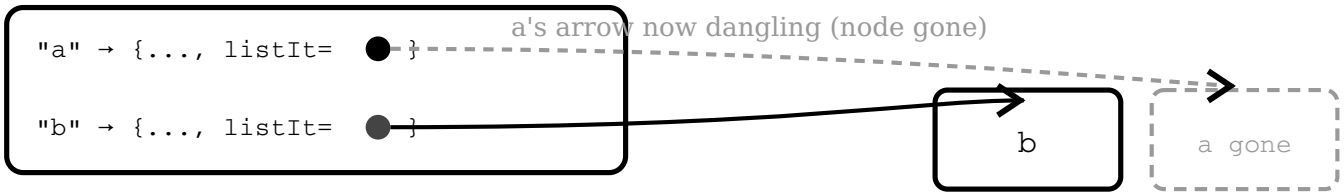
Result: two arrows now — "a"'s listIt → back box, "b"'s listIt → front box. List = [b, a]. Reply: `OK`

STEP 3 — GET a → returns "1", then touch("a")

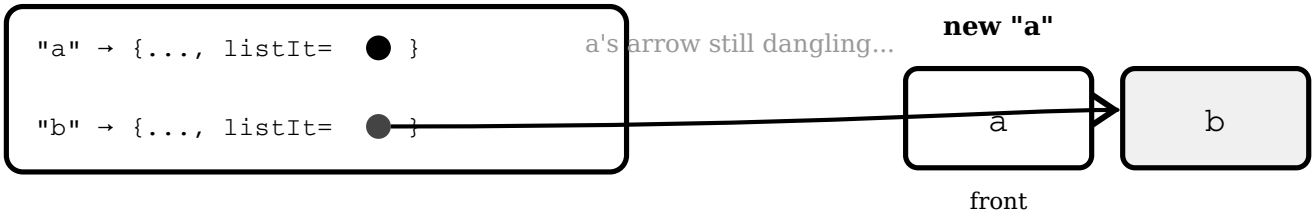
GET finds "a", returns its value, then calls `touch("a")` to mark it recent. Watch "a"'s arrow move.

```
// inside touch("a"):
auto it = data_.find("a");
usageList_.erase(it->second.listIt);    // 3a: follow a's arrow, delete that node
usageList_.push_front("a");             // 3b: make a NEW "a" node at front
it->second.listIt = usageList_.begin(); // 3c: re-point a's arrow to the new node
```

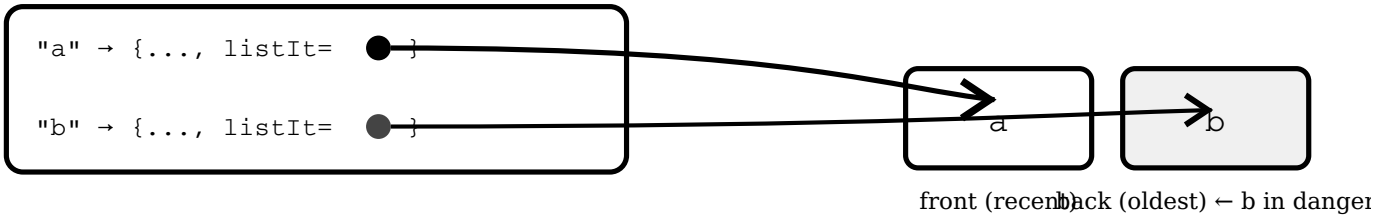
3a — erase via listIt: list [b, a] → [b] (a's old node deleted, its arrow now dangling)



3b — push_front("a"): list [b] → [a, b] (brand-new "a" node at front)



3c — listIt = begin(): re-point a's arrow to the NEW front node → fixed, O(1) ready



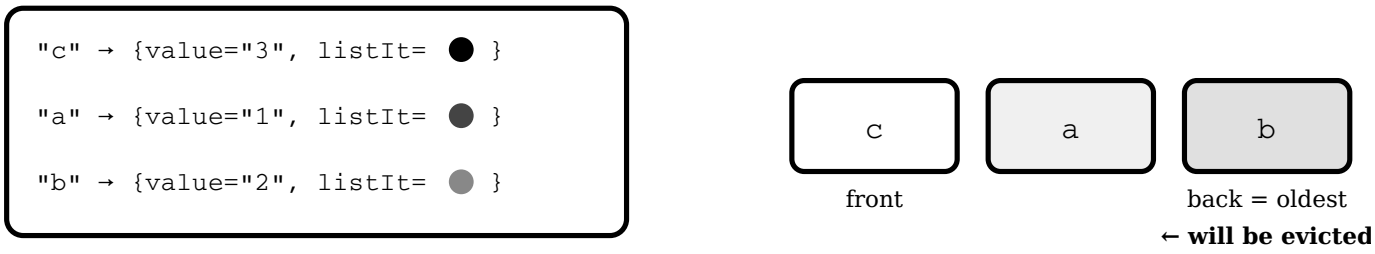
Result: List = [a, b]. "a"'s arrow re-pointed to front. Now **"b" is the oldest** (back) — nobody touched it. Reply: `1`

> **WHY RE-POINT (3C)?**
>
> The old "a" room was demolished and a new "a" room built at the front. The register still listed the old room number — pointing at rubble. Step 3c writes the new room number. Miss it → next visit walks into rubble (crash).

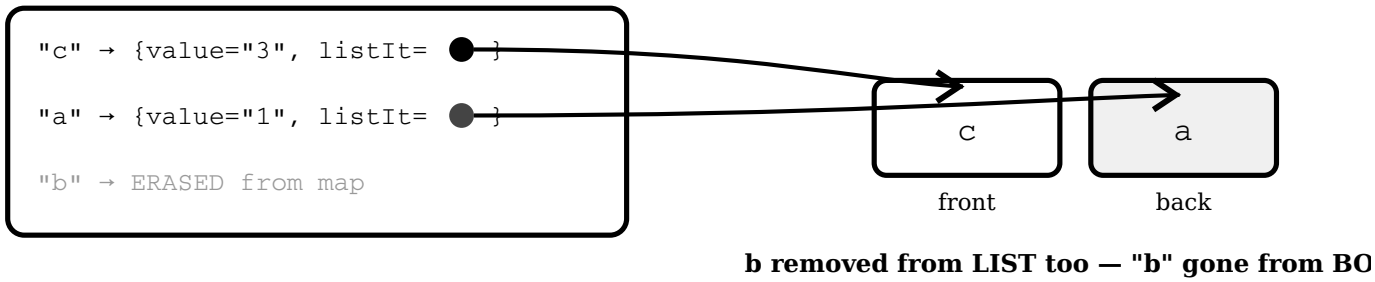
STEP 4 — SET c 3 → 3 keys > capacity 2 → EVICT

```
usageList_.push_front("c");        // list [c, a, b]
Entry e; e.value="3"; e.listIt = usageList_.begin();
data_["c"] = e;                    // map now has 3 keys
evictIfNeeded();                   // size 3 > 2  -> EVICT
  // inside evictIfNeeded():
  std::string oldest = usageList_.back();  // "b" (the back)
  usageList_.pop_back();                   // remove b from LIST
  data_.erase(oldest);                     // remove b from MAP
  // prints: [lru] evicted: b
```

After push_front("c"): list [c, a, b], map has a, b, c (OVERFULL)



evictIfNeeded: back()="b" → pop_back (LIST) + erase (MAP). b removed from BOTH.



Result: List = [c, a], map = {c, a}. Server prints `[lru] evicted: b`. Then `GET b` → `(nil)`.

Why "b"? GET a (Step 3) made "a" recent. "b" was never touched again → it sat at the back → evicted.

SUMMARY — full state after every command

Command	MAP (key → value, listIt)	LIST	Reply
`SET a 1`	a→("1", →a)	[a]	OK
`SET b 2`	a→("1",→a), b→("2",→b)	[b, a]	OK
`GET a`	a→("1",→a'), b→("2",→b)	[a, b]	1
`SET c 3`	c→("3",→c), a→("1",→a) **[b evicted]**	[c, a]	OK + evict b
`GET b`	{c, a} (b gone)	[c, a]	(nil)

THE ONE RULE

A key lives in BOTH map and list. listIt is the arrow linking them. So every operation touches BOTH: add → list + map; touch → move in list + re-point listIt; remove → erase from list + map. Forget one side → dangling arrow → crash.

IN-MEMORY KV STORE — MASTER INTERVIEW GUIDE

Phase 1→6 end-to-end. ISSUE → OVERCOME → SOLUTION → DRY RUN → Q&A. For FAANG / hardware / fintech / IB / HFT / payments systems rounds.

What you built (one breath)

A concurrent in-memory key-value store in C++20 over TCP, supporting SET / GET / DEL / EXPIRE, with TTL expiry (lazy + active), per-shard O(1) LRU eviction, and **16-way sharded locking** — benchmarked at **2.45M ops/sec, 6.3× throughput and 11× lower P99** versus a single global lock.

Phase	Problem it solves	Core technique
1 — TCP Server	talk to clients over the network	POSIX sockets: socket/bind/listen/accept/recv/send
2 — Parsing	raw bytes are meaningless	tokenize + validate into a Command struct
3 — Storage	actually remember data	unordered_map (hash table)
4 — TTL	keys must expire by time	chrono deadline + lazy & active expiry thread
5 — LRU	memory must stay bounded	map + doubly-linked list + cached iterator, O(1)
6 — Concurrency	many clients, no lock bottleneck	thread-per-client + 16-shard locking + benchmark

PHASE 1 — TCP SERVER (sockets)

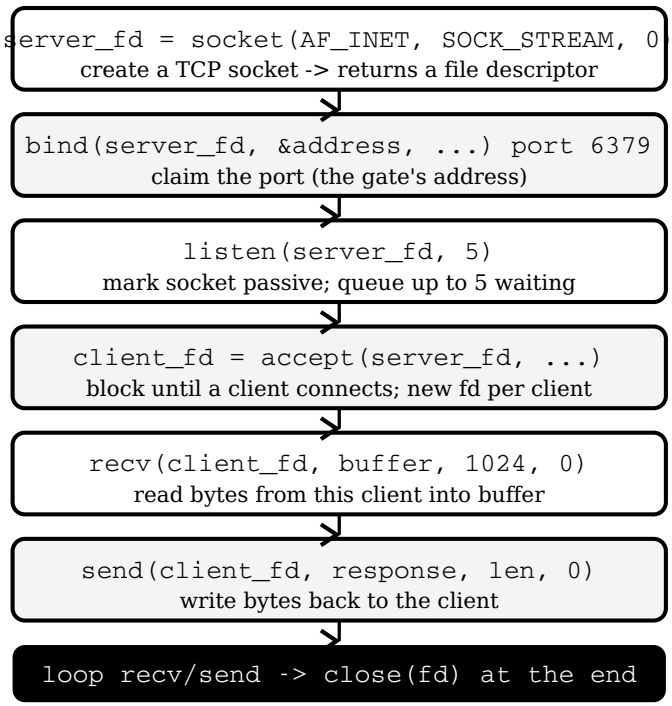
ISSUE

A key-value store is useless if clients cannot reach it. We need a way to accept connections over the network and exchange bytes.

OVERCOME (the thinking)

The OS exposes the network through **sockets**. A server socket must be created, bound to a port, set to listen, then it accepts client connections. Each accepted connection is a separate file descriptor over which we `recv` and `send` bytes.

SOLUTION — socket lifecycle



Call	Meaning
`socket(AF_INET, SOCK_STREAM, 0)`	IPv4 + TCP (stream) socket; returns a file descriptor (an int handle)
`bind`	attach the socket to port 6379 on this machine
`listen(fd, 5)`	make it passive; backlog of 5 pending connections
`accept`	blocks; returns a NEW fd dedicated to one client
`recv / send`	read / write bytes on that client fd

server_fd = the gate (accepts everyone). client_fd = a private line to ONE client. They are different fds.

Q&A — Phase 1

Q. What is a socket and what does the fd represent?

A socket is an OS endpoint for network communication; `socket()` returns a **file descriptor** — a small integer the kernel maps to the connection's state. We read/write bytes through that fd as if it were a file.

> **Counter:** "*Why SOCK_STREAM not SOCK_DGRAM?*" → *SOCK_STREAM is TCP — reliable, ordered, connection-based; a KV protocol needs request/response integrity, which UDP (DGRAM) doesn't guarantee.*

Q. accept() blocks — what does "blocking" mean and why does it matter?

A blocking call suspends the thread until data/connection is ready. `accept` sleeps until a client connects; `recv` sleeps until bytes arrive. Blocking I/O is simple but means one thread can serve one client at a time — the exact limitation Phase 6 removes with threads.

> **Counter:** "*Alternatives to blocking?*" → *Non-blocking sockets with `epoll/select` (event-driven, one thread many connections) or async I/O. I used thread-per-client for clarity; epoll is the scaling alternative.*

Q. Why send `bytes` from recv, not `sizeof(buffer)`?

`recv` returns how many bytes actually arrived; the buffer is 1024 but only those `bytes` are real data. Sending `sizeof(buffer)` would transmit 1024 bytes including garbage/zeros.

PHASE 2 — COMMAND PARSING

ISSUE

recv gives us a raw string like `"SET name shivam\n"`. The server has no idea what it means — it's just bytes.

OVERCOME

Split the string into tokens on whitespace, look at the first token to identify the command, validate the token count, and fill a structured `Command` object the rest of the server can act on.

SOLUTION — the Command shape + parser

```
enum class CommandType { SET, GET, DEL, EXPIRE, UNKNOWN };
struct Command {
    CommandType type  = CommandType::UNKNOWN;
    std::string key, value;
    int  seconds = 0;          // for EXPIRE
    bool valid   = false;
    std::string error;
};
Command parse(const std::string& input) {
    std::stringstream ss(input);
    std::string token; std::vector<std::string> tokens;
    while (ss >> token) tokens.push_back(token);     // split on spaces
    if (tokens.empty()) { cmd.error="empty"; return cmd; }
    if (tokens[0]=="SET") { if(tokens.size()!=3){...err} cmd.type=SET;
        cmd.key=tokens[1]; cmd.value=tokens[2]; cmd.valid=true; }
    ... GET/DEL need 2 tokens, EXPIRE needs 3 (key + seconds via stoi) ...
}
```

DRY RUN — "SET name shivam"

`input = "SET name shivam\n"`

trim newline -> "SET name shivam" (15 chars)

stringstream splits on spaces:

"SET"	"name"	"shivam"
tokens[0]=command	tokens[1]=key	tokens[2]=value

"SET" matched, size==3 valid -> fill Command:

```
type=SET key="name" value="shivam"
seconds=0 valid=true error=""
```

Step	State
`recv -> buffer`	"SET name shivam\n" + zeros, bytes=16
`string(buffer, bytes)`	"SET name shivam\n"
`trim \n`	"SET name shivam"
`tokens`	["SET","name","shivam"]
`Command`	{SET, "name", "shivam", valid=true}

Q&A — Phase 2

Q. Why a Command struct instead of acting on the string directly?

Separation of concerns: parsing is one job, execution another. The struct is a validated, typed representation; the dispatcher just switches on `cmd.type`. It also centralizes validation (token counts, errors) in one place.

> **Counter:** "*Real protocols?*" → *Redis uses RESP, a length-prefixed binary protocol; mine is a simplified whitespace text protocol — same idea, simpler framing.*

Q. `const std::string&` for the input — why?

Pass by const reference: no copy of the string (efficiency) and a compile-time promise not to mutate it (safety). Standard for read-only parameters.

PHASE 3 — STORAGE (hash map)

ISSUE

The server understands commands but stores nothing — SET is acknowledged then forgotten. We need real storage with fast lookup.

OVERCOME

A hash table gives average O(1) insert/lookup/delete. Wrap an `unordered_map` in a KVStore class so storage is encapsulated and can be extended later (TTL, LRU).

SOLUTION

```
void KVStore::set(const string& k, const string& v){ data_[k]=v; }   // insert/overwrite
bool KVStore::get(const string& k, string& out){
    auto it = data_.find(k);             // find, NOT data_[k] (which would create blank)
    if (it == data_.end()) return false; // not found
    out = it->second; return true;       // fill out-param, report found
}
bool KVStore::del(const string& k){ return data_.erase(k) > 0; }    // count removed
```

get() returns a bool (found?) AND writes the value into an out-parameter reference — a function returns one value, so the bool is returned and the string is filled by reference.

DRY RUN — SET / GET / DEL

SET name shivam

```
{name -> shivam}
```

GET name -> "shivam"

```
find -> it->second
```

DEL name

```
erase -> { }
```

GET name (after DEL) -> find == end() -> return false -> client sees (nil)

Q&A — Phase 3

Q. Why `find()` and not `data_[key]` in get?

`operator[]` on a missing key **inserts** a default-constructed entry — a silent bug for a read. `find()` only looks; it returns `end()` if absent and never mutates the map.

> **Counter:** "*Complexity of unordered_map?*" → *Average O(1) for insert/find/erase; worst case O(n) under heavy hash collisions or rehashing.* `std::map` *would be O(log n) but ordered — we don't need order, so the hash table wins.*

Q. What is the out-parameter pattern and why use it?

A function returns one value; `get` must report two things — "found?" and "the value." So it returns the bool and writes the value into a reference argument. The caller passes a string; on success it's filled. Cleaner alternatives exist (`std::optional`), but the out-param is explicit and zero-cost.

PHASE 4 — TTL EXPIRY (time + threads)

ISSUE

Keys live forever. A cache needs keys that auto-expire after a time (sessions, OTPs, rate-limit windows).

OVERCOME

Store a deadline with each value. Remove dead keys two ways: **lazy** (check on read) for correctness, and **active** (a background thread) for memory hygiene. Because a second thread now touches the map, guard it with a **mutex**.

SOLUTION — Entry gains a deadline; map value becomes Entry

```
struct Entry { string value; steady_clock::time_point expireAt; bool hasExpiry=false; };
unordered_map<string, Entry> data_;   mutex mutex_;   atomic<bool> running_;   thread cleaner_;

bool expire(key, sec){ lock_guard g(mutex_);
    it->second.expireAt = steady_clock::now() + seconds(sec); it->second.hasExpiry=true; }

bool get(key, out){ lock_guard g(mutex_);
    Entry& e = it->second;
    if (e.hasExpiry && now() > e.expireAt){ data_.erase(it); return false; }  // LAZY
    out=e.value; return true; }

void cleanupLoop(){ while(running_){ sleep_for(1s); lock_guard g(mutex_);  // ACTIVE
    for(it=data_.begin(); it!=data_.end(); ){
        if(expired) it=data_.erase(it); else ++it; } } }
```

Piece	Why
`steady_clock`	monotonic clock; immune to wall-clock/date changes — reliable for deadlines
`hasExpiry flag`	most keys are permanent; flag false skips the time check (&& short-circuits)
`mutex_ + lock_guard`	two threads (client + cleaner) share data_; RAII lock prevents data races
`atomic<bool> running_`	single flag read by the loop, set false in destructor — atomic fits one value
`it = erase(it)`	erase invalidates the iterator; erase returns the next valid one (no ++it in header)

DRY RUN — Entry state over time (start 12:00:00)

Time	Command	Entry / map state	Reply
00	`SET name shivam`	name {value:shivam, hasExpiry:false}	OK
02	`EXPIRE name 5`	name {shivam, expireAt:12:00:07, hasExpiry:true}	OK
04	`GET name`	now 04 < 07 -> alive	shivam
09	`GET name`	now 09 > 07 -> LAZY erase -> { }	(nil)

If no GET happens, the background cleaner thread removes it within ~1s of expiry (ACTIVE) — memory never leaks dead keys.

Q&A — Phase 4

Q. Lazy vs active expiry — why both?

Lazy (on `get`) is cheap and guarantees a read never returns a dead value, but a key nobody reads lingers. Active (background sweep) reclaims memory regardless of access. Together: correct reads + bounded memory. Redis uses the same combination.

> **Counter:** "*Does the cleaner block clients for the whole sweep?*" → *It holds the lock only while scanning; in the sharded version (Phase 6) it locks one shard at a time, so clients keep working on the other shards.*

Q. Why a mutex for the map but an atomic for the flag?

Atomics make a single primitive's read/write indivisible — perfect for a `bool`. A hash map is a composite structure (buckets, sizes, rehashing); you cannot make it atomic, so it needs a mutex. Right tool per data type.

> **Counter:** "*Why steady_clock not system_clock?*" → `system_clock` *can jump if the user changes the date;* `steady_clock` *only moves forward, so a 5-second TTL is always 5 real seconds.*

Q. Explain the iterator-invalidation bug in the cleanup loop.

Erasing a map element invalidates its iterator. If the for-header did `++it` after an erase, it would advance a dangling iterator — undefined behaviour. The fix: `it = data_.erase(it)` (erase returns the next valid iterator) on deletion, and `++it` only when not erasing. The for-header has no increment.

Q. How is the cleanup thread stopped cleanly?

The destructor sets `running_ = false` (the loop's condition) then calls `join()`, which waits for the thread to finish its current iteration and exit. Without `join`, the thread could touch a destroyed object — a crash.

PHASE 5 — LRU EVICTION (map + list + iterator)

ISSUE

Memory is finite. Permanent keys (no TTL) accumulate until the process crashes. TTL alone can't help — it only removes keys that were given a deadline.

OVERCOME

Cap the store. When full, evict the **Least Recently Used** key. Track recency with a doubly-linked list (front = recent, back = oldest). To move/evict in O(1), each map entry caches its own iterator into that list.

SOLUTION

```
struct Entry { string value; ...; list<string>::iterator listIt; };  // cached position
list<string> usageList_;   size_t capacity_;

void touch(key){                                  // mark as most-recent, O(1)
    usageList_.erase(it->second.listIt);           // remove via cached iterator
    usageList_.push_front(key);                    // re-insert at front
    it->second.listIt = usageList_.begin(); }      // save NEW position

void evictIfNeeded(){ if (data_.size() <= capacity_) return;
    string oldest = usageList_.back();             // back = least recently used
    usageList_.pop_back(); data_.erase(oldest); }  // remove from BOTH

set(k,v){ if exists: update + touch;
    else: push_front(k); e.listIt=begin(); data_[k]=e; evictIfNeeded(); }
get(k): ... touch(k); // a read counts as "used"
```

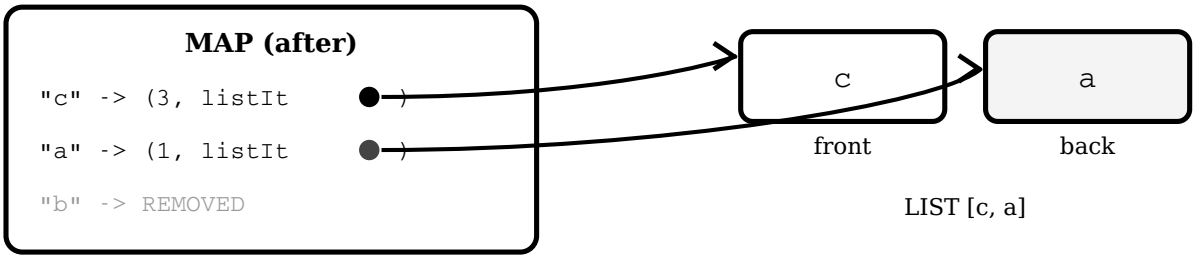
DRY RUN — capacity=2, map + list + iterator together

1) SET a 1 `MAP {a:(1, listIt●)} LIST [a]`

2) SET b 2 (full) `MAP {a, b} LIST [b, a] front=b recent`

3) GET a -> touch(a) `LIST [a, b] now b is oldest (back)`

4) SET c 3 -> 3 keys > cap 2 -> EVICT back = b



[lru] evicted: b -> GET b now returns (nil)

Why b, not a? GET a (step 3) made "a" recent; "b" was never touched again, so it sat at the back and was evicted. listIt is the arrow from each map entry to its list node — it makes touch/evict O(1) (no list search).

Q&A — Phase 5

Q. Why is your LRU O(1)? Walk me through it.

Recency lives in a doubly-linked list; each map entry caches its own list iterator. On access, erase via that iterator and push to front — both O(1), no scan. On overflow, pop the back — O(1). The cached iterator is the key: without it, locating the node would be O(n).

> **Counter:** "*Why not a vector or array?*" → *Moving an element to the front of a vector is O(n) (shifts) and invalidates iterators;* `std::list` *gives*

> *O(1) splice and stable iterators, which is exactly what the cached-iterator trick needs.*

Q. TTL already deletes keys — why also LRU?

They target different things. TTL removes keys by **time**, and only those given a deadline. LRU removes keys by **space** — any key, including permanent ones, when memory is full. A store full of permanent keys would crash with TTL alone; LRU bounds it.

Q. Map and list both hold the key — how do you keep them in sync?

Every mutating path touches both: add → push to list + insert in map; touch → move in list + update the cached iterator; remove (evict/del/expiry) → erase from list + erase from map. Touching only one side leaves a dangling iterator or a ghost node — so the invariant is "always both."

PHASE 6 — CONCURRENCY & SHARDED LOCKING

ISSUE

Two problems. (1) The server handled ONE client at a time. (2) Even with threads, every operation took ONE global lock — unrelated keys queued, killing throughput under load.

OVERCOME

(1) **Thread-per-client**: an accept loop spawns a worker thread per connection, all sharing one store. (2) **Sharded locking**: split the keyspace into 16 shards, each with its own map, list, and mutex; route keys by `hash % 16`. Different shards = different locks = true parallelism.

SOLUTION (1) — thread-per-client

```
void Server::run(){ KVStore store(100);            // ONE shared store
    while(true){ int fd = accept(...);
        thread(&Server::handleClient, this, fd, std::ref(store)).detach(); } }
void Server::handleClient(int fd, KVStore& store){
    while(true){ recv -> parse -> store.set/get/... -> send } close(fd); }
```

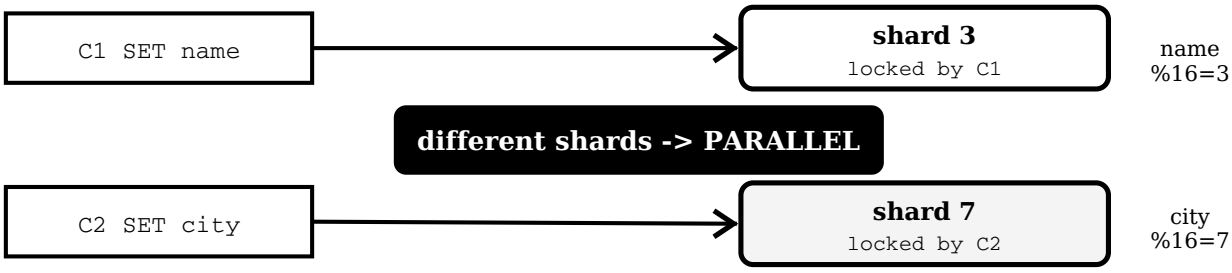
`std::ref(store)`	share the SAME store by reference (not a copy) across all threads
`.detach()`	let the worker run independently; manager returns to accept the next client

SOLUTION (2) — the single-lock problem, drawn



SOLUTION (2) — 16 shards + routing

```
struct Shard { unordered_map<string,Entry> data; list<string> usageList; mutex mutex; };
Shard shards_[16];
size_t getShardIndex(key){ return std::hash<std::string>{}(key) % 16; }   // routing
void set(k,v){ Shard& s = shards_[getShardIndex(k)];   // pick shard
    lock_guard g(s.mutex);                              // lock ONLY this shard
    ... same map+LRU logic on s.data / s.usageList ... }
```



Same key -> same hash -> same shard, always: that's why GET finds what SET stored. Different keys usually hit different shards -> run in parallel. Same shard -> one waits (rare, ~1/16).

SOLUTION (3) — benchmark (the proof)

```
8 threads x 100,000 ops (50% SET / 50% GET), direct on the store (no network).
vector<vector<long long>> latencies;  // per-thread -> no shared-vector race
time each op; ops/sec = total/time; sort all -> P50 (median), P99 (tail)
```

Metric	1 shard (single lock)	16 shards	Gain
Throughput	388,349 ops/sec	**2,453,987 ops/sec**	**6.3×**
Total time (800K ops)	2.06 s	0.33 s	6.3× faster
Latency P50	2,462 ns	**827 ns**	3× lower
Latency P99	153,237 ns	**13,605 ns**	**11× lower**

Q&A — Phase 6 (the heavy round)

Q. What is lock contention and how does sharding reduce it?

Contention is threads competing for the same lock — while one holds it, others stall. One global lock means every operation contends with every other. Sharding gives 16 independent locks; two operations contend only if they hit the same shard (~1/16), so most run in parallel. Measured: 6.3× throughput, 11× lower P99.

> **Counter:** "*Does adding shards always help?*" → *No — gains flatten once shard count exceeds hardware threads, and memory grows (each shard carries its own map/list/mutex). It scales contention down, not beyond physical parallelism.*

Q. Why a mutex per shard, not a lock-free map?

Lock-free hash maps exist but are complex and bug-prone (memory reclamation, ABA). Per-shard mutexes give most of the parallelism with far less risk. I used `std::atomic` only where it fits a single value (the cleanup flag). Lock-free is the stated future extension.

> **Counter:** "*HFT would demand lock-free.*" → *Correct — for sub-microsecond hot paths you'd go lock-free or per-core sharded with no shared state; my design is the lock-based stepping stone and I can articulate the next step.*

Q. Hot-key skew — all traffic on one shard?

That shard becomes a hotspot and contention returns for those keys — a known limit of static hash sharding. Mitigations: more shards, a better hash, or consistent hashing with virtual nodes; adaptive resharding for skew is future work.

Q. Is the benchmark fair? How do you know sharding caused the gain?

The only variable changed between runs is `SHARD_COUNT` (1 vs 16) — identical code, identical workload, same machine, same threads and key distribution. That isolation is what makes the 6.3× attributable to sharding and nothing else.

> **Counter:** "*Per-thread latency vectors — why?*" → *If all threads pushed into one shared vector, that vector itself would be a contended, racing structure — corrupting both the data and the measurement. Per-thread vectors need no lock; I merge and sort them at the end.*

Q. Why report P99, not average latency?

Averages hide tail stalls; a few very slow ops barely move the mean but define the user-visible worst case. Under one lock the unlucky request waits behind everyone (P99 153µs); sharding removed that stall (13.6µs). Production SLAs are written on tail latency, which is why it's the headline.

Q. How would this scale across machines?

Same hash-partitioning idea, one level up: route keys to server nodes by hash (consistent hashing for elastic membership) — exactly how Redis Cluster and Memcached scale out. My in-process sharding is the single-node analogue, which makes it a natural foundation for a distributed store.

CROSS-CUTTING / SYSTEM-DESIGN ROUND

Q. Walk me through the full path of "SET name shivam" end to end.

Client sends bytes -> `recv` into a 1024 buffer (16 real bytes) -> build a string of exactly those bytes -> trim the trailing newline -> `parse` splits into tokens, matches "SET", validates 3 tokens, fills a Command -> dispatcher routes to `store.set` -> `getShardIndex("name")` = hash%16 picks a shard -> lock that shard -> insert into the shard's map, push to its LRU list, store the list iterator, evict if over capacity -> unlock -> build "OK\n" -> `send` back.

Q. What are the time complexities across the system?

Parse: O(L) in input length. Hash routing: O(key length). Map insert/find/erase: average O(1). LRU touch/evict: O(1) via the cached iterator. Active expiry sweep: O(n) per second across all shards but amortized and per-shard locked. So the hot path (SET/GET) is effectively O(1).

Q. Where is the memory overhead?

Each entry stores value + a time_point + a bool + a list iterator. Each shard carries a map, a list, and a mutex — 16 of each. Sharding trades a small fixed memory overhead for large concurrency gains; with thousands of keys per shard, per-key overhead dominates and shard overhead is negligible.

Q. What would you improve with more time? (shows maturity)

Persistence (append-only log / snapshot), RESP-style binary protocol, epoll-based event loop to scale past thread-per-client, lock-free or per-core structures for tail latency, consistent hashing for skew and multi-node scale-out, and ThreadSanitizer/fuzz testing in CI.

Q. What was the hardest concept and one concrete bug you fixed?

Hardest: reasoning about correctness under concurrency — why a whole container needs a mutex (not an atomic), and iterator invalidation. Concrete bug: in the new-key path of SET I initially forgot to set the entry's `listIt`, leaving a dangling iterator that crashed on the next touch; fixed by pushing to the list and storing `begin()` before inserting into the map.

CLOSING PITCH: "Concurrent in-memory KV store, built from scratch in C++. Sharded locking cut contention — 6.3× throughput and 11× lower P99 versus a single global lock. I can defend every data structure, every lock, and every number."